

Lab 1

Przedmiot: Wstęp do Informatyki

Struktura wiersza poleceń:

Większość poleceń w wierszu poleceń ma tę samą podstawową strukturę:

polecenie [opcja(e)/parametr(y)...] [argument(y)...]

Rozważmy następujące polecenie jako przykład:

```
$ ls -l /home
```

Wyjaśnijmy przeznaczenie każdego elementu polecenia:

Command (Polecenie)

Przykład programu `ls` uruchomionego przez użytkownika w powyższym przykładzie.

Option(s)/Parameter(s) (Opcja(e)/parametr(y))

“Przełącznik” (“switch”) w pewien sposób modyfikuje zachowanie polecenia, na przykład `-l` w powyższym przykładzie. Dostęp do opcji można uzyskać w postaci krótkiej i długiej. Na przykład `-l` jest identyczne z zapisem `--format=long`.

Można również łączyć kilka opcji, a w przypadku krótkiej formy litery można zwykle pisać razem. Na przykład następujące polecenia robią to samo:

```
$ ls -al
```

```
$ ls -a -l
```

```
$ ls --all --format=long
```

Argument(s) (Argument(y))

Dodatkowe informacje wymagane przez program, takie jak nazwa pliku lub ścieżka, na przykład `/home` w powyższym przykładzie.

Jedyną obowiązkową częścią tej struktury jest samo polecenie. Ogólnie wszystkie inne elementy są opcjonalne, ale program może wymagać określenia pewnych opcji, parametrów lub argumentów.

Typy poleceń

Powłoka obsługuje dwa typy poleceń:

Polecenia wewnętrzne (wbudowane)

Te polecenia są częścią samej powłoki, a nie samodzielnymi programami. Takich poleceń jest około 30. Ich głównym celem jest wykonywanie zadań wewnątrz powłoki (np. `cd`, `set`, `export`).

Polecenia zewnętrzne

Te polecenia znajdują się w poszczególnych plikach. Zwykle są to programy binarne lub skrypty. Kiedy uruchamiane i wykonywane jest polecenie, które nie jest poleceniem wbudowanym w powłokę, powłoka używa zmiennej PATH w celu wyszukania pliku wykonywalnego o tej samej nazwie co polecenie. Oprócz programów, które są instalowane za pomocą menedżera pakietów, użytkownicy mogą również tworzyć własne polecenia zewnętrzne.

Polecenie `type` pokazuje, jakiego typu jest dane polecenie:

```
$ type echo
```

```
echo is a shell builtin
```

```
$ type man
```

```
man is /usr/bin/man
```

Quoting (znaki cudzysłowu)

Jako użytkownik Linuxa będziesz musiał tworzyć lub manipulować plikami lub zmiennymi na wiele różnych sposobów. Jest to łatwe podczas pracy z krótkimi nazwami plików i pojedynczymi wartościami. Sytuacja staje się jednak bardziej skomplikowana, gdy wykorzystywane są m.in. spacje, znaki specjalne i zmienne. Powłoki zapewniają funkcję zwaną "Quoting", która zawiera takie dane w różnych typach cudzysłowów (" ", ' '). W Bashu istnieją trzy rodzaje cudzysłowów:

- Podwójne cudzysłowy
- Pojedyncze cudzysłowy
- Znaki ucieczki

Podwójne cudzysłowy

Podwójne cudzysłowy informują powłokę, że tekst między cudzysłowami ("...") jest traktowany jako zwykłe znaki. Wszystkie znaki specjalne tracą znaczenie, z wyjątkiem `$` (znak dolara), `\` (backslash - ukośnik odwrotny) i ``` (backquote - cudzysłów odwrotny), dzięki czemu można nadal używać zmiennych, podstawiania poleceń i funkcji arytmetycznych.

Na przykład podwójne cudzysłowy nie mają wpływu na podstawienie zmiennej `$USER`:

```
$ echo I am $USER
```

```
I am tom
```

```
$ echo "I am $USER"
```

```
I am tom
```

Z drugiej strony znak spacji traci znaczenie jako separator argumentów:

```
$ touch new file
```

```
$ ls -l
```

```
-rw-rw-r-- 1 tom students 0 Oct 8 15:18 file
```

```
-rw-rw-r-- 1 tom students 0 Oct 8 15:18 new
```

```
$ touch "new file"
```

```
$ ls -l
```

```
-rw-rw-r-- 1 tom students 0 Oct 8 15:19 new file
```

Jak widać, polecenie touch w pierwszym przykładzie tworzy dwa oddzielne pliki, ponieważ polecenie interpretuje dwa ciągi jako oddzielne argumenty. W drugim przykładzie, polecenie interpretuje oba ciągi jako jeden argument, czyli tylko jedną nazwę pliku, dlatego tworzy tylko jeden plik. Najlepszą praktyką jest jednak unikanie znaku spacji w nazwach plików. Zamiast tego można użyć podkreślenia (_) lub kropki (.).

Pojedyncze cudzysłowy

Pojedyncze cudzysłowy nie mają wyjątków takich jak podwójne cudzysłowy. Unieważniają one jakiegokolwiek specjalne znaczenie każdego znaku. Weźmy jeden z powyższych przykładów:

```
$ echo I am $USER
```

```
I am tom
```

Po zastosowaniu pojedynczych cudzysłówów zobaczysz inny wynik:

```
$ echo 'I am $USER'
```

```
I am $USER
```

Polecenie wyświetla teraz dokładny ciąg znaków bez zastępowania zmiennej.

Znaki ucieczki

Możemy użyć znaków ucieczki, aby usunąć specjalne znaczenie znaków z Basha. Wracając do zmiennej środowiskowej \$USER:

```
$ echo $USER
```

```
carol
```

Widzimy, że domyślnie zawartość zmiennej jest wyświetlana w terminalu. Jeśli jednak poprzedzimy znak dolara znakiem backslasha (\), to specjalne znaczenie znaku dolara zostanie anulowane. To z kolei nie pozwoli Bashowi rozwinąć wartości zmiennej w celu uwzględnienia nazwy użytkownika osoby wykonującej polecenie, zatem zamiast tego Bash zinterpretuje nazwę zmiennej dosłownie:

```
$ echo \$USER
```

```
$USER
```

Jeśli pamiętasz, podobny wynik możemy osiągnąć za pomocą pojedynczych cudzysłówów, ponieważ zapewniają one, że treść zawarta między cudzysłowami jest dokładnie taka, jaka znajduje się między tymi pojedynczymi cudzysłowami. Jednak znak ucieczki działa inaczej, mówiąc Bashowi, aby zignorował jakiegokolwiek specjalne znaczenie, jakie może mieć znak po nim.

2. Podstawowe polecenia w Bash'u

- **pwd** - wypisuje na ekran aktualny katalog
- **ls** - wypisuje listę plików oraz podkatalogów danego katalogu, przydatne opcje -l, -h, -a. Domyślnie dotyczy aktualnego katalogu (.).
- **cd** - przechodzi do podanego katalogu. Bez podania katalogu przejdzie do katalogu domowego
- **who** - wypisuje listę użytkowników zalogowanych na danym systemie
- **id** - nazwa, identyfikator oraz grupa użytkownika
- **mkdir nazwa_katalogu** - tworzy folder
- **touch nazwa_pliku** - tworzy plik (gdy plik istnieje zmienia datę ostatniej modyfikacji)
- **rm [target]** - usuwa plik, z flagą -r usuwa katalog razem z wszystkimi podkatalogami oraz plikami
- **mv [target] [destination]** - przynosi podany plik lub katalog (-r), służy także do zmiany nazwy
- **cp [target] [destination]** - kopiuje podany plik lub katalog
- **cal** - wypisuje kalendarz
- **ps** - wypisuje informacje o procesach
- **kill PID** - po podaniu identyfikatora procesu (PID) kończy proces
- **chmod** - zmiana praw dostępu
- **echo** - wypisanie argumentów na ekran - przydatne do wypisywania zmiennych środowiskowych (o tym później)
- **cat** - połączenie oraz wypisanie zawartości przekazanych strumieni (np. pliki) na ekran
- **wc** - zlicza słowa, linie, znaki z pliku
- **tr** - zamienia frazy w plikach

- **grep wzor [plik]** - wyszukuje frazę wypisując tylko linie które spełniają warunek, przyjmuje plik lub strumień tekstowy
- **less plik** - wypisanie na ekran zawartości pliku w trybie interaktywnym
- **tree [katalog]** - wypisuje strukturę katalogów
- **file plik** - wypisuje informacje o pliku
- **tail [OPCJE] [PLIK]** - służy do wyświetlania ostatnich wierszy (lub, bajtów) danego pliku
- **head [OPCJE] [PLIK]** - wyświetlanie początkowej części pliku
- **history [OPCJE]** - wyświetla historie poleceń
- **find** - Przeszukuje system plików pod kątem nazw, które pasują do szeregu kryteriów wyboru

Zadania:

1. Zalogować się jako root (administrator) i w katalogu /home utworzyć następującą strukturę katalogów korzystając z samej powłoki/wiersza poleceń:

[\]

+home

+imie nazwisko

+Moje Dokumenty

+Adam

Czyli: w katalogu /home utworzyć katalog o nazwie "Adam", oraz o nazwie składającej się z własnego imienia i nazwiska, a następnie w katalogu tym utworzyć katalog "Moje Dokumenty".

Należy również na końcu zweryfikować prawidłowe utworzenie struktury katalogów wykorzystując polecenie wyświetlające zawartość danego wskazanego katalogu, jak również na końcu wykorzystując polecenie wyświetlające strukturę katalogów (tj. wew. katalogu /home) w formie tzw. drzewa.

2. Wyszukaj w systemie operacyjnym (korzystając tylko i wyłącznie z samego systemu operacyjnego GNU/Linux), jaki przełącznik należy użyć w ramach polecenia "ls", aby pliki zostały wyświetlone posortowane wg. czasu modyfikacji (poczynając od najnowszych).

3. Utwórz 5 plików ponumerowanych od 1 do 5 z prefiksem game (game1, game2, ...) za pomocą jednego polecenia i używając rozszerzenia nawiasów klamrowych w Bash.
4. Usuń wszystkie 5 plików, które właśnie utworzyłeś jednym poleceniem, używając innego znaku specjalnego.
5. Utworzyć w katalogu /home/{imie nazwisko} ukryty plik o nazwie dokument.txt, a następnie wyświetlić zawartość tego katalogu w taki sposób, aby zobaczyć także ukryty plik, a na końcu należy go również usunąć.
6. Sprawdzić czas ostatniej modyfikacji, jak również ostatniego odczytu pliku /etc/adduser.conf
7. Wpisać w powłoce/wierszu poleceń w szybki sposób (bez konieczności przepisywania znaku po znaku) i dający 100% pewności bezbłędności wpisania, ścieżkę: /usr/share/doc/netcat-traditional/examples/scripts/probe
8. Wiedząc, że dana komenda rozpoczyna się na od znaków "net", znaleźć w szybki sposób wszystkie komendy rozpoczynające się od tychże znaków.
9. Będąc zalogowany jako root (administrator):
Wyświetlić (w powłoce/wierszu poleceń):
 - całą zawartość pliku /etc/passwd (korzystając z odpowiedniego skrótu klawiszowego przeglądnąć całą wyświetloną zawartość pliku)
 - całą zawartość pliku /etc/shadow ekran po ekranie
 - ostatnich 5 wierszy pliku /etc/passwd
 - pierwszych 7 wierszy pliku /etc/passwd
10. Będąc zalogowany jako root (administrator):
Przywrócić (w powłoce) przedostatnie wydane polecenie w wierszu poleceń systemu operacyjnego. Następnie wyświetlić wszystkie wydane do tej pory polecenia w wierszu poleceń systemu operacyjnego.
11. Będąc zalogowany jako root (administrator):
 1. Utworzyć (korzystając z powłoki/wiersza poleceń) plik /home/{imie nazwisko}/Moje Dokumenty/plik.txt oraz plik /home/{imie nazwisko}/Moje Dokumenty/plik2.txt
 2. W pliku /home/{imie nazwisko}/Moje Dokumenty/plik.txt korzystając z edytora tekstu wpisać tekst: "To jest mój plik tekstowy"
 3. Jak można szybciej wykonać powyższe zadanie dot. pliku plik.txt?
12. Będąc zalogowany jako root (administrator):
 - 1) Wyszukać w całym systemie plików pliki oraz katalogi posiadające w swojej nazwie wyrażenie "menu"
 - 2) Wyszukać w całym systemie plików pliki oraz katalogi których nazwa rozpoczyna się od wyrażenia "doc"

3) Wyszukać w katalogu /bin pliki oraz katalogi, których nazwa składa się z 4 znaków, gdzie pierwszy znak to litera "a", lub "b", lub "c", lub "d"

4) Wyszukać w katalogu /bin pliki oraz katalogi, których nazwa składa się z 4 znaków, gdzie pierwszy znak to jakikolwiek inny niż litera "a", lub "b", lub "c", lub "d"